Abstract

In universities and colleges, students have to select and enroll in various courses every semester. The traditional way of doing this using physical paper forms, long queues, and manual entry in spreadsheets causes many problems such as lost records, slow processing, schedule clashes, and seat confusion. To solve these everyday problems, this project introduces the **Course Enrollment Portal**, a modern web-based application built using the full-stack MERN architecture (MongoDB, Express.js, React.js, and Node.js).

The main goal of the portal is to give college students a single, easy-to-use website where they can explore all available course offerings, enroll with a single click, and track their learning progress step-by-step. Instead of using rigid traditional SQL tables that require fixed schemas and complex table joins, we used **MongoDB**, a high-performance NoSQL document-oriented database. MongoDB stores all course details, student profiles, and enrollments in flexible JSON documents, making it very easy to manage dynamic course information and update seat availability in real time.

The system features secure login and signup with encrypted passwords and JSON Web Tokens (JWT) for safe access. It offers 24 academic courses categorized into 6 major branches: Computer Science, AI & Machine Learning, Data Science, Web Development, Cloud & DevOps, and Cybersecurity. An automatic seat capacity check prevents over-enrollment, and an interactive progress tracker allows students to record completion milestones (25%, 50%, 75%, and 100%), helping students stay motivated and complete their studies successfully.

Introduction

In modern colleges and universities, education has expanded beyond traditional degree programs. Every semester, students have to select and enroll in different core subjects, elective courses, and practical skill-based modules. Because colleges offer a wide variety of courses across different academic departments, managing student course enrollment has become one of the most essential day-to-day administrative tasks for institutions.

In the past, academic institutions managed course registrations manually using physical paper forms. Students had to stand in long queues outside administrative offices, fill out registration sheets by hand, and wait days for approval. College clerks had to manually type all this data into spreadsheets or ledgers. This manual method was slow, caused frequent mistakes in student names and course codes, and created huge confusion during peak admission weeks.

To eliminate these difficulties, universities are now adopting online course enrollment portals. A web-based portal allows students to view the complete course catalog, read syllabus outlines, check credit points, and see available seats from their laptops or mobile phones. Students can enroll in their preferred courses with just a single click at any time, without needing to visit departmental offices physically or fill out paper forms.

To build an efficient and responsive portal, this project uses the MERN technology stack (MongoDB, Express.js, React.js, and Node.js). Instead of using old, rigid SQL databases with fixed tables and complex joins, we chose MongoDB as our database. MongoDB stores all student profiles, course information, and enrollment details in flexible JSON-like documents. This allows the system to easily handle real-time seat updates, support diverse course structures, and process multiple student enrollments at the exact same moment without crashing.

The portal provides a clean, fast interface where students can browse 24 courses divided across 6 main branches, such as Computer Science, AI/ML, and Web Development. It includes automatic seat capacity checks to prevent over-enrollment and features a step-by-step progress tracker (25%, 50%, 75%, and 100%). This gives students a clear visual view of their academic journey, helps teachers monitor classroom attendance, and makes the entire university enrollment process simple, transparent, and error-free.

Problem Statement

Managing course registrations manually or with basic spreadsheets creates multiple problems that hurt college operations and cause student frustration:

- o Heavy Paperwork and Lost Time: Filling out paper forms by hand wastes valuable time for both students and university staff.
- o Seat Over-Allocation: Without a live database, more students register than the classroom can physically hold.
- o Lack of Real-Time Information: Students cannot see how many seats are remaining in a course before submitting their choices.
- o Rigid Database Limitations: Traditional SQL databases make it difficult to add new elective courses with different credit points or weekly modules without redesigning the entire database.
- o No Progress Tracking: After enrolling, students have no central dashboard to see how much of the course they have finished (e.g., 25%, 50%, 75%, 100%) or what grade they achieved.

Therefore, there is a clear need for an automated, web-based Course Enrollment Portal with a flexible MongoDB database that centralizes courses, prevents seat over-allocation, and provides transparent progress tracking.

Objective

o To build a simple and responsive Course Enrollment Portal where students can browse, filter, and enroll in university courses from any device.

o To use MongoDB for database storage so that student accounts, course details, and enrollments are stored in flexible JSON documents.

o To implement secure User Authentication using encrypted passwords (bcrypt) and JSON Web Tokens (JWT) for student and instructor accounts.

o To manage course seat limits automatically and prevent students from registering once a course reaches maximum capacity.

o To provide a Milestone Progress Tracker (25%, 50%, 75%, 100%) so students can monitor their course completion and earn final grades.

o To offer Course Management (CRUD) so authorized teachers and administrators can easily add, edit, or remove course offerings.

Literature Review

Over the last twenty years, educational software has evolved through several stages. In the beginning, academic institutions used basic desktop applications connected to traditional relational database management systems (RDBMS) like MySQL or Oracle.

Early researchers like Silberschatz explained that relational databases are well suited for strict tabular records where every row has the same fixed columns. However, modern university curricula are diverse. Different courses have different credit points, lab requirements, weekly lessons, and prerequisites. Representing all of these in rigid SQL tables requires many separate tables linked by complex JOIN queries, which makes the system slow to load and difficult to update.

To overcome the limitations of traditional relational databases, modern web applications started adopting NoSQL document-oriented databases like MongoDB. As explained by computer science author Kristina Chodorow, MongoDB stores data in documents rather than rigid tables. In an enrollment portal, each course document can easily hold its title, branch, credit points, instructor name, and lesson outline together in one clean structure.

Another major advantage of MongoDB is its high performance under heavy traffic. During college registration week, hundreds of students try to enroll at the exact same moment. In traditional SQL databases, simultaneous writes often cause table locking, slow response times, and system crashes. MongoDB solves this by using atomic update operators (such as $inc to increase the enrolled student counter), ensuring that seats are updated accurately without locking up the database.

In recent years, full-stack JavaScript development using the MERN stack (MongoDB, Express, React, Node.js) has become the industry standard for building interactive web applications. As noted by web developers like Ethan Brown, using JavaScript across both the frontend and backend eliminates the need to translate data between different languages. The event-driven, non-blocking nature of Node.js ensures that incoming enrollment requests are handled quickly with minimal server memory.

Furthermore, building the backend as a RESTful API provides a clean separation between the user interface and database storage. This means the same backend can serve a desktop website, a mobile browser, or even a future mobile app without rewriting server code.

On the client side, React.js has revolutionized how users interact with websites. Traditional multi-page websites reload the entire page every time the user clicks a link. In contrast, React builds Single-Page Applications (SPAs) where only the updated part of the screen is changed. In our course portal, this allows students to filter courses by branch or enroll with instant visual feedback, creating a fast and smooth user experience.

Research in educational psychology shows that visual progress tracking helps students stay motivated and finish their courses. When students can see their progress moving step-by-step from 25% to 50%, 75%, and 100%, they feel a sense of accomplishment and are much less likely to drop out. Traditional university portals only recorded whether a student was enrolled or not, offering no ongoing support during the semester.

Security is another essential requirement for any student management system. Modern applications use JSON Web Tokens (JWT) for secure, stateless login sessions. When combined with bcrypt password hashing, student credentials remain safe from unauthorized access. The backend checks the digital signature of the token on every request, ensuring that only logged-in students can view their personal enrollments.

In summary, existing research and modern software development practices prove that replacing manual paper forms and rigid SQL tables with a modern MERN stack portal and MongoDB database creates a faster, safer, and far more user-friendly course enrollment experience.

System Overview

The Course Enrollment Portal is designed using a clean, three-tier client-server architecture. This structure divides the application into three independent layers:

o Frontend (Presentation Layer): Built with React.js, Vite, and Tailwind CSS. It runs in the student's browser and displays course cards, branch filter buttons, login/signup forms, and progress tracking bars.

o Backend (Application Layer): Built with Node.js and the Express.js framework. It acts as the brain of the portal. It handles API requests (/api/auth, /api/courses, /api/enrollments), validates user tokens, checks seat limits, and talks to the database.

o Database (Data Persistence Layer): Powered by MongoDB and Mongoose ODM. It safely stores user accounts, course information, and student enrollment records in JSON documents.

1. Data Flow and User Interaction

The operational workflow follows a simple request-and-response pattern:

1. When a student opens the website, the React frontend asks the backend server for available courses.
2. The backend queries MongoDB and returns the 24 courses as JSON data. React then displays these courses as interactive cards on the screen.
3. When the student clicks 'Enroll Now', the frontend sends an enrollment request containing the course ID and student login token.
4. The backend checks if the student is already enrolled and whether seats are available. If valid, it saves the new enrollment in MongoDB, increases the enrolled count on the course by 1, and sends a success message back to the student.

2. Major Modules of the System

The Course Enrollment Portal consists of four interconnected functional modules:

o User Authentication Module: Manages user registration and login for students and instructors. It encrypts passwords using bcrypt and issues secure JWT tokens. It ensures that students can only access their own dashboard and enrollments.

o Course Catalog & Branch Filtering Module: Displays 24 courses organized across 6 main academic branches: Computer Science, AI & Machine Learning, Data Science, Web Development, Cloud & DevOps, and Cybersecurity. Students can filter courses with a single click, view instructor names, credits, duration, and see live seat availability.

o Enrollment & Seat Capacity Module: Handles student registration with automatic seat verification. Each course has a maximum capacity (e.g., 50 seats). The system prevents any student from enrolling once all seats are filled, completely avoiding over-enrollment.

o Milestone Progress Tracking Module: In the 'My Courses' tab, students can see all the courses they have joined. They can update their learning progress through four milestones: 25% (Foundation), 50% (Core Modules), 75% (Advanced Concepts), and 100% (Course Completed). Reaching 100% automatically awards an 'A' grade.

o Course Management Module (CRUD): Allows authorized instructors and admins to create new courses, edit course details, adjust seat limits, or remove old courses through simple popup forms.

This modular structure makes the portal easy to maintain, simple to upgrade, and fast to run on any computer.

Requirements

1. User Roles and Characteristics

The Course Enrollment Portal is built for three main groups of users:

o Students: College students who want to explore course electives, check credits and instructor details, enroll in courses with one click, and track their weekly progress.

o Faculty Instructors: Academic teachers who review enrolled student lists, monitor student progress percentages, and manage course syllabi.

o Department Administrators: University coordinators who create new course offerings, define seat capacities, assign instructors, and view enrollment statistics.

2. Hardware Requirements

The system has lightweight hardware requirements and works smoothly on standard computers:

o Client-Side Hardware (For Students & Teachers):

- Computer, laptop, tablet, or smartphone with an active internet connection.
- Processor: Dual-Core Intel Core i3 / AMD Ryzen 3 or any modern mobile processor.
- RAM: Minimum 2 GB (4 GB recommended).
- Storage: 500 MB free hard disk space for browser caching.
- Web Browser: Google Chrome, Microsoft Edge, Mozilla Firefox, or Safari.

o Server-Side Hardware (For Hosting Backend & Database):

- Processor: Quad-Core Intel Core i5 / AMD Ryzen 5 or Cloud VPS.
- RAM: Minimum 4 GB (8 GB recommended for heavy traffic).
- Storage: 20 GB SSD storage space for MongoDB database and system files.
- Network: Standard broadband or campus Wi-Fi connection.

3. Functional Requirements

Functional requirements define the core features and operations performed by the portal:

- FR-1: User Registration and Login: The portal shall allow students and instructors to sign up using name, email, and password. It must validate email uniqueness and issue JWT tokens on successful login.
- FR-2: Course Catalog Display: The portal shall present a clean catalog of 24 courses displaying course title, branch, credit points, duration, instructor, and seat availability.
- FR-3: Branch-Wise Course Filtering: Students shall be able to filter courses across 6 branches (Computer Science, AI & ML, Data Science, Web Development, Cloud & DevOps, Cybersecurity) with one click.
- FR-4: Single-Click Course Enrollment: Logged-in students shall be able to enroll in an open course by clicking 'Enroll Now'.
- FR-5: Duplicate Enrollment Prevention: The system shall check if a student is already registered for a course and block duplicate enrollments.
- FR-6: Real-Time Seat Capacity Enforcement: The system shall increase the enrolled student counter by 1 upon enrollment and disable registration when capacity reaches zero.
- FR-7: Milestone Progress Updates: Students shall be able to update their learning progress across 25%, 50%, 75%, and 100% milestones.
- FR-8: My Courses Dashboard: The portal shall provide a personal dashboard showing all enrolled courses, live progress bars, and completion statuses.
- FR-9: Course CRUD Management: Instructors shall be able to add new courses, update details, or delete courses easily.

4. Non-Functional Requirements

Non-functional requirements describe system quality, reliability, and security standards:

o Speed and Performance: All page loads and API requests shall complete in under 200 milliseconds under normal network conditions.

o Password Security: All user passwords must be hashed using bcrypt (10 salt rounds) before saving to MongoDB so they can never be stolen.

o Reliability and Uptime: The system shall maintain 99.5% uptime during active semester enrollment windows.

o Simple and Clean Interface: The user interface must be modern, attractive, and easy to navigate on both desktop and mobile screens.

o Error Handling: The portal must show clear, friendly popup notifications whenever an error happens (e.g., 'Course is full' or 'Already enrolled').

5. MongoDB Database Collections

The database uses three main collections in MongoDB to store all project data:

o Users Collection: Stores student, instructor, and admin accounts. Fields include _id, name, email (unique), password (hashed), role (student/instructor/admin), department, bio, and avatarUrl.

o Courses Collection: Stores the 24 academic courses. Fields include _id, title, code (unique), description, instructor, category (branch), level, duration, credits, capacity, enrolledCount, and rating.

o Enrollments Collection: Maps which student enrolled in which course. Fields include _id, user (ObjectId ref User), course (ObjectId ref Course), status (Active/Completed/Dropped), progress (0-100), completedLessons, and grade. A compound unique index prevents duplicate enrollments.

Tools and Technologies Used

Category	Technology	Purpose and Role in Project
Database Engine	MongoDB (v7.0)	NoSQL document database used to store users, courses, and enrollments in JSON format.
ODM Library	Mongoose (v8.0)	Schema-based library for connecting Node.js with MongoDB and enforcing data validation.
Server Runtime	Node.js (v20 LTS)	Asynchronous JavaScript server environment used to run the backend REST API.
Web Framework	Express.js (v4.19)	Lightweight backend framework used to create API routes, middleware, and controllers.
Frontend Library	React.js (v18.3)	JavaScript library used to build the responsive, component-based user interface.
Build Tool	Vite (v5.4)	Fast modern development server and build tool with instant hot module reloading.
Styling Framework	Tailwind CSS (v3.4)	Utility-first CSS framework used for clean, responsive dark-mode styling.
Authentication	JWT (jsonwebtoken)	Stateless token standard used to secure user login sessions across requests.
Password Hashing	bcryptjs (v2.4)	Cryptographic library used to hash and salt user passwords for high security.
Database GUI	MongoDB Compass	Visual management tool used to inspect collections, documents, and verify enrollments.
API Testing Tool	Postman	Tool used to test and verify all backend API endpoints before connecting the frontend.

All these modern technologies work together seamlessly using JavaScript across the entire system, making development fast, reliable, and easy to maintain.

Code

```
// ============================================================
// 1. MongoDB Database Connection (server.js)
// ============================================================
const mongoose = require('mongoose');
const MONGODB_URI = process.env.MONGODB_URI ||
  'mongodb://127.0.0.1:27017/course_enrollment_db';

mongoose.connect(MONGODB_URI)
  .then(() => console.log('MongoDB Connected to course_enrollment_db'))
  .catch((err) => console.error('MongoDB Connection Error:', err));

// ============================================================
// 2. User Data Model (models/User.js)
// ============================================================
const UserSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true },
  role: {
    type: String,
    enum: ['student', 'instructor', 'admin'],
    default: 'student'
  },
  department: { type: String, default: 'Computer Science' },
  bio: { type: String, default: 'Passionate student learner.' },
  avatarUrl: { type: String, default: '' }
}, { timestamps: true });

module.exports = mongoose.model('User', UserSchema);

// ============================================================
// 3. Course Data Model (models/Course.js)
// ============================================================
const CourseSchema = new mongoose.Schema({
  title: { type: String, required: true, trim: true },
  code: { type: String, required: true, unique: true, uppercase: true },
  description: { type: String, required: true },
  instructor: { type: String, required: true },
  category: {
    type: String,
    required: true,
    enum: ['Computer Science', 'Data Science & AI', 'Web Development',
           'UI/UX Design', 'Cloud & DevOps', 'Cybersecurity']
  },
  level: { type: String, enum: ['Beginner', 'Intermediate', 'Advanced'] },
  duration: { type: String, default: '8 Weeks' },
  credits: { type: Number, default: 3 },
  capacity: { type: Number, default: 50 },
```

```
// ============================================================
// 4. Enrollment Data Model (models/Enrollment.js)
// ============================================================
const EnrollmentSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  course: { type: mongoose.Schema.Types.ObjectId, ref: 'Course', required: true },
  status: {
    type: String,
    enum: ['Active', 'Completed', 'Dropped'],
    default: 'Active'
  },
  progress: { type: Number, min: 0, max: 100, default: 0 },
  completedLessons: [{ type: String }],
  grade: { type: String, default: 'In Progress' },
  enrolledAt: { type: Date, default: Date.now }
}, { timestamps: true });

// Compound unique index to strictly prevent duplicate course enrollment
EnrollmentSchema.index({ user: 1, course: 1 }, { unique: true });
module.exports = mongoose.model('Enrollment', EnrollmentSchema);

// ============================================================
// 5. Course Enrollment API Controller (routes/enrollments.js)
// ============================================================
const express = require('express');
const router = express.Router();
const Enrollment = require('../models/Enrollment');
const Course = require('../models/Course');
const auth = require('../middleware/auth');

// POST /api/enrollments - Enroll student in course with seat validation
router.post('/', auth, async (req, res) => {
  try {
    const { courseId } = req.body;
    if (!courseId) return res.status(400).json({ message: 'Course ID required.' });

    const course = await Course.findById(courseId);
    if (!course) return res.status(404).json({ message: 'Course not found.' });

    // Check if student is already enrolled
    const exists = await Enrollment.findOne({ user: req.user.id, course: courseId });
    if (exists) return res.status(400).json({ message: 'Already enrolled.' });

    // Verify classroom seat capacity
    if (course.enrolledCount >= course.capacity) {
      return res.status(400).json({ message: 'Course is at maximum capacity.' });
    }

    // Create new enrollment record
    const enrollment = new Enrollment({
```

```
// ============================================================
// 6. Progress Milestone Update API (routes/enrollments.js)
// ============================================================
// PUT /api/enrollments/:id/progress - Update learning milestone
router.put('/:id/progress', auth, async (req, res) => {
  try {
    const { progress, completedLessons } = req.body;
    const enrollment = await Enrollment.findOne({
      _id: req.params.id,
      user: req.user.id
    });
    if (!enrollment) return res.status(404).json({ message: 'Record not found.' });

    if (progress !== undefined) {
      const p = Math.min(100, Math.max(0, Number(progress)));
      enrollment.progress = p;

      // Automatically award completion status and Grade upon 100%
      if (p === 100) {
        enrollment.status = 'Completed';
        enrollment.grade = 'A';
      } else {
        enrollment.status = 'Active';
      }
    }
    if (completedLessons) enrollment.completedLessons = completedLessons;

    await enrollment.save();
    const updated = await Enrollment.findById(enrollment._id).populate('course');
    res.json({ message: 'Progress updated successfully!', enrollment: updated });
  } catch (err) {
    res.status(500).json({ message: 'Error updating progress.' });
  }
});

// ============================================================
// 7. MongoDB Aggregation Queries for Analytical Reporting
// ============================================================
// Query 1: Get all enrolled courses for logged-in student with course metadata
const getStudentCourses = async (studentId) => {
  return await Enrollment.find({ user: studentId })
    .populate('course')
    .sort({ createdAt: -1 });
};

// Query 2: Aggregate total courses and seat occupancy by academic branch
const getBranchStats = async () => {
  return await Course.aggregate([
    {
      $group: {
```

Result and Output

The Course Enrollment Portal was successfully implemented and tested. The web portal works smoothly and performs all core operations without any errors.

The homepage provides a clean, modern dashboard displaying 24 academic courses categorized into 6 branches. Students can easily filter courses by branch (such as Computer Science, AI/ML, or Web Development), inspect course credits, duration, and instructor names, and check live available seats before enrolling.

[Click and Paste Figure 1 Screenshot: Course Enrollment Portal Dashboard]

Fig 1: Course Enrollment Portal Dashboard and Course Catalog

The database tier was verified using MongoDB Compass and backend API testing. When a student clicks enroll, a new enrollment document is created in MongoDB immediately, and the course seat counter updates accurately. Compound indexes prevent any student from enrolling in the same course twice.

The milestone progress tracker was also tested across all four stages: 25%, 50%, 75%, and 100%. When a student reaches 100%, the portal automatically awards an 'A' grade and marks the enrollment as Completed, demonstrating full end-to-end functionality.

[Click and Paste Figure 2 Screenshot: MongoDB Course Enrollments and Progress View]

Fig 2: MongoDB Course Enrollments and Progress Tracking View

Advantages

o No Paperwork: The entire course enrollment process is digital. Students do not need to stand in long queues or fill out paper forms.

o Real-Time Seat Capacity: Students can see exactly how many seats are left before clicking enroll, completely avoiding over-enrollment.

o Schema Flexibility with MongoDB: New courses, disciplines, and lesson modules can be added easily without redesigning database tables.

o Clear Progress Tracking: Students can track their learning through 25%, 50%, 75%, and 100% milestones, keeping them motivated.

o Fully Responsive Design: The website works smoothly and looks great on desktop PCs, laptops, tablets, and mobile phones.

Future Scope

o AI-Powered Course Recommendations: Use machine learning algorithms to suggest elective courses based on each student's interests and past performance.

o Automated Prerequisite Checks: Automatically check student transcripts to confirm that introductory courses are completed before allowing advanced enrollment.

o Online Payment Gateway: Add secure payment integration (UPI, Net Banking, Credit Cards) for lab fees and elective certification charges.

o Downloadable Course Certificates: Automatically generate verified PDF completion certificates when a student completes 100% of a course.

CONCLUSION

The design, development, and deployment of the **Course Enrollment Portal** successfully solves the major problems of traditional paper-based course registration in colleges. By utilizing the modern **MERN full-stack architecture (MongoDB, Express.js, React.js, Node.js)**, we built a fast, scalable, and easy-to-use web application for students and faculty.

Choosing **MongoDB** as our database management system proved to be an excellent decision. Storing data in flexible JSON documents eliminated the rigid structure, difficult schema changes, and complex table JOINs of traditional SQL databases. Atomic updates allow live seat counts and student enrollment records to update instantly and safely, even during peak registration hours.

The React.js frontend gives students a clean, modern interface where they can explore 24 courses across 6 academic branches, filter by subject with one click, register instantly, and monitor their learning progress through visual milestone bars. Strong password hashing (bcrypt) and JSON Web Tokens ensure that all student records remain completely safe and secure.

In conclusion, this project shows how modern web technologies and NoSQL databases can make college academic management simpler, faster, and much more transparent for everyone involved.

BIBLIOGRAPHY

1. Chodorow, Kristina. (2013). *MongoDB: The Definitive Guide: Powerful and Scalable Data Storage*. 2nd ed. O'Reilly Media.
2. Banker, Kyle, et al. (2016). *MongoDB in Action: Covers MongoDB version 3.0*. 2nd ed. Manning Publications.
3. Silberschatz, Abraham, Henry F. Korth, and S. Sudarshan. (2020). *Database System Concepts*. 7th ed. McGraw-Hill Education.
4. Banks, Alex and Eve Porcello. (2020). *Learning React: Modern Patterns for Developing React Applications*. 2nd ed. O'Reilly Media.
5. Herron, David. (2020). *Node.js Web Development: Server-side development with Node 14 and Express*. 5th ed. Packt Publishing.
6. Brown, Ethan. (2019). *Web Development with Node and Express: Leveraging the JavaScript Stack*. 2nd ed. O'Reilly Media.
7. Official MongoDB Documentation. (2026). *Mongoose ODM and Aggregation Pipeline Reference Guide*. Available at: https://www.mongodb.com/docs/.
8. Official React Documentation. (2026). *React - A JavaScript Library for Building User Interfaces*. Available at: https://react.dev/.
9. World Wide Web Consortium (W3C). (2023). *Web Content Accessibility Guidelines (WCAG) 2.1*. W3C Recommendation.

Index

Sr. NO.	Contents	Page No.
1	Abstract	1
2	Introduction	2-3
3	Problem Statement	3
4	Objective	4
5	Literature Review	4-6
6	System Overview	7-8
7	Requirements	9-11
8	Tools and Technologies used	12
9	Code	13-15
10	Results and Output	16-17
11	Advantages	18
12	Future Scope	18
13	Conclusion	19
14	Bibliography	20